Cluster Analysis

What Is Cluster Analysis?

Clustering is the process of grouping a set of data objects into multiple groups or clusters so that objects within a cluster have high similarity, but are very dissimilar to objects in other clusters. Dissimilarities and similarities are assessed based on the attribute values describing the objects and often involve distance measures. Clustering as a data mining tool has its roots in many application areas such as biology, security, business intelligence, and Web search.

The set of clusters resulting from a cluster analysis can be referred to as **clustering**. Clustering is useful in that it can lead to the discovery of previously unknown groups within the data.

Cluster analysis has been widely used in many applications such as business intelligence, image pattern recognition, Web search, biology, and security.

Few Applications of Clustering

1. Biology: taxonomy of living things: kingdom, phylum, class, order, family, genus and species
2. Information retrieval: document clustering
3. Land use: Identification of areas of similar land use in an earth observation database
4. Marketing: Help marketers discover distinct groups in their customer bases, and then use this
5. knowledge to develop targeted marketing programs
6. City-planning: Identifying groups of houses according to their house type, value, and
7. geographical location
8. Climate: understanding earth climate, find patterns of atmospheric and ocean

<u>Clustering as a Preprocessing Tool</u>

Summarization: Preprocessing for regression, PCA, classification, and association analysis

Compression: Image processing: vector quantization

Finding K-nearest Neighbors: Localizing search to one or a small number of clusters

Outlier detection: Outliers are often viewed as those "far away" from any cluster.

The following are typical requirements of clustering in data mining.

Scalability: Many clustering algorithms work well on small data sets containing fewer than several hundred data objects; however, a large database may contain millions or even billions of objects, particularly in Web search scenarios. Clustering on only a sample of a given large data set may lead to biased results. Therefore, highly scalable clustering algorithms are needed.

Ability to deal with different types of attributes: Many algorithms are designed to cluster numeric (interval-based) data. However, applications may require clustering other data types, such as binary, nominal (categorical), and ordinal data, or mixtures of these data types. Recently, more and more applications need clustering techniques for complex data types such as graphs, sequences, images, and documents.

Discovery of clusters with arbitrary shape: Many clustering algorithms determine clusters based on Euclidean or Manhattan distance measures. Algorithms based on such distance measures tend to find spherical clusters with similar size and density. However, a cluster could be of any shape. Consider sensors, for example, which are often deployed for environment surveillance. Cluster analysis on sensor

readings can detect interesting phenomena. We may want to use clustering to find the frontier of a running forest fire, which is often not spherical. It is important to develop algorithms that can detect clusters of arbitrary shape.

Requirements for domain knowledge to determine input parameters: Many clustering algorithms require users to provide domain knowledge in the form of input parameters such as the desired number of clusters. Consequently, the clustering results may be sensitive to such parameters. Parameters are often hard to determine, especially for high-dimensionality data sets and where users have yet to grasp a deep understanding of their data. Requiring the specification of domain knowledge not only burdens users, but also makes the quality of clustering difficult to control.

Ability to deal with noisy data: Most real-world data sets contain outliers and/or missing, unknown, or erroneous data. Sensor readings, for example, are often noisy—some readings may be inaccurate due to the sensing mechanisms, and some readings may be erroneous due to interferences from surrounding transient objects. Clustering algorithms can be sensitive to such noise and may produce poor-quality clusters. Therefore, we need clustering methods that are robust to noise.

Incremental clustering and insensitivity to input order: In many applications, incremental updates (representing newer data) may arrive at any time. Some clus tering algorithms cannot incorporate incremental updates into existing clustering structures and, instead, have to recompute a new clustering from scratch. Clustering algorithms may also be sensitive to the input data order. That is, given a set of data objects, clustering algorithms may return dramatically different clusterings depending on the order in which the objects are presented. Incremental clustering algorithms and algorithms that are insensitive to the input order are needed.

Capability of clustering high-dimensionality data: A data set can contain numerous dimensions or attributes. When clustering documents, for example, each keyword can be regarded as a dimension, and there are often thousands of keywords. Most clustering algorithms are good at handling low-dimensional data such as data sets involving only two or three dimensions. Finding clusters of data objects in a high dimensional space is challenging, especially considering that such data can be very sparse and highly skewed.

Constraint-based clustering: Real-world applications may need to perform clus tering under various kinds of constraints. Suppose that your job is to choose the locations for a given number of new automatic teller machines (ATMs) in a city. To decide upon this, you may cluster households while considering constraints such as the city's rivers and highway networks and the types and number of customers per cluster. A challenging task is to find data groups with good clustering behavior that satisfy specified constraints.

Interpretability and usability: Users want clustering results to be interpretable, comprehensible, and usable. That is, clustering may need to be tied in with spe cific semantic interpretations and applications. It is important to study how an application goal may influence the selection of clustering features and clustering methods.

<u>Overview of Basic Clustering Methods :</u>

There are many clustering algorithms in the literature. It is difficult to provide a crisp categorization of clustering methods because these categories may overlap so that a method may have features from

several categories. Nevertheless, it is useful to present a relatively organized picture of clustering methods. In general, the major fundamental clustering methods can be classified into the following categories.

Partitioning methods: Given a set of n objects, a partitioning method constructs k partitions of the data, where each partition represents a cluster and $k \leq n$. That is, it divides the data into k groups such that each group must contain at least one object. In other words, partitioning methods conduct one-level partitioning on data sets. The basic partitioning methods typically adopt exclusive cluster separation. That is, each object must belong to exactly one group. This requirement may be relaxed, for example, in fuzzy partitioning techniques.

Most partitioning methods are distance-based. Given k, the number of partitions to construct, a partitioning method creates an initial partitioning. It then uses an iterative relocation technique that attempts to improve the partitioning by moving objects from one group to another. The general criterion of a good partitioning is that objects in the same cluster are "close" or related to each other, whereas objects in different clusters are "far apart" or very different. There are various kinds of other criteria for judging the quality of partitions.

Hierarchical methods: A hierarchical method creates a hierarchical decomposition of the given set of data objects. A hierarchical method can be classified as being either agglomerative or divisive, based on how the hierarchical decomposition is formed. The agglomerative approach, also called the bottom-up approach, starts with each object forming a separate group. It successively merges the objects or groups close to one another, until all the groups are merged into one (the topmost level of the hierarchy), or a termination condition holds. The divisive approach, also called the top-down approach, starts with all the objects in the same cluster. In each successive iteration, a cluster is split into smaller clusters, until eventually each object is in one cluster, or a termination condition holds.

Hierarchical clustering methods can be distance-based or density- and continuity- based. Various extensions of hierarchical methods consider clustering in subspaces as well.

Density-based methods: Most partitioning methods cluster objects based on the distance between objects. Such methods can find only spherical-shaped clusters and encounter difficulty in discovering clusters of arbitrary shapes. Other clustering methods have been developed based on the notion of density. Their general idea is to continue growing a given cluster as long as the density (number of objects or data points) in the "neighborhood" exceeds some threshold. For example, for each data point within a given cluster, the neighborhood of a given radius has to contain at least a minimum number of points. Such a method can be used to filter out noise or outliers and discover clusters of arbitrary shape. Density-based methods can divide a set of objects into multiple exclusive clus ters, or a hierarchy of clusters. Typically, density-based methods consider exclusive clusters only, and do not consider fuzzy clusters. Moreover, density based methods can be extended from full space to subspace clustering.

Different Types of Clusters:

Type of Cluster	Key Idea	Shape/Structure	Common Algorithms
Well-Separated	Clearly separated groups	Circular / Isolated	K-Means, K-Medoids
Center-Based	Points close to a centroid	Spherical	K-Means, EM
Prototype-Based	Represented by a prototype object/centroid	Compact or spherical	K-Means, SOM
Contiguity-Based	Neighbor-connected points	Irregular / Chain-like	Hierarchical (Single-Link)
Density-Based	Dense regions separated by sparse areas	Arbitrary	DBSCAN, OPTICS
Shared-Property (Conceptual)	Common semantic or conceptual property	Not geometric	COBWEB, Conceptual Clustering
Graph-Based	Highly connected subgraphs in a graph	Network-like	MST, Spectral Clustering

Partitioning Methods:

The simplest and most fundamental version of cluster analysis is partitioning, which organizes the objects of a set into several exclusive groups or clusters. To keep the problem specification concise, we can assume that the number of clusters is given as background knowledge. This parameter is the starting point for partitioning methods.

Formally, given a data set, D, of n objects, and k, the number of clusters to form, a partitioning algorithm organizes the objects into k partitions ($k \leq n$), where each partition represents a cluster. The clusters are formed to optimize an objective partitioning criterion, such as a dissimilarity function based on distance, so that the objects within a cluster are "similar" to one another and "dissimilar" to objects in other clusters in terms of the data set attributes.

K-Means Algorithm

Algorithm: *k*-means. The *k*-means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

Input:

k: the number of clusters,

D: a data set containing *n* objects.

Output: A set of *k* clusters.

Method:

(1) arbitrarily choose *k* objects from *D* as the initial cluster centers;

(2) repeat

(3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;

(4) update the cluster means, that is, calculate the mean value of the objects for each cluster;

(5) **until** no change;

Handling Key Issues in K-Means Clustering:

Handling Empty Clusters

- Problem: During iterations, a cluster may have no data points assigned.
- Effect: Centroid has no members → disrupts clustering.
- Solutions:
  - Reassign the empty centroid to the point farthest from other centroids.
  - Restart clustering if necessary.

Handling Outliers

- **Problem:** Outliers distort the centroid since K-Means uses **mean values**.
- **Effect:** Cluster center shifts away from the dense region.
- **Solutions:**
  - **Preprocess data** to remove or limit outliers.
  - Use **K-Medoids** instead of K-Means (less sensitive to outliers).

Reducing SSE with Preprocessing and Cluster Refinement

- **SSE (Sum of Squared Errors):** Measures total variance within clusters.
- **Goal:** Minimize SSE for better clustering.

A. Preprocessing Steps

1. **Feature Scaling:** Standardize or normalize features.
2. **Outlier Removal:** Remove extreme values before clustering.
3. **Dimensionality Reduction:** Use PCA to remove redundant or irrelevant features.

B. Cluster Refinement Techniques

1. **Split Cluster:**
   - Divide a large cluster into two smaller clusters if it shows high internal variance.
   - Helps reduce SSE by making clusters more compact.
2. **Introduce a New Cluster Centroid:**
   - Add a new centroid in a dense or poorly represented area of data.
   - Allows the algorithm to capture subgroups that were previously underrepresented.
3. **Disperse a Cluster:**
   - If points in a cluster are widely spread, disperse them to reduce intra-cluster variance.

- Can involve reassigning points to nearby clusters or creating new clusters.

4. **Merge Two Clusters:**
    - Combine two small or overlapping clusters into one if SSE decreases.
    - Prevents unnecessary fragmentation of clusters.

Updating Centroids Incrementally

- **Problem:** Centroids need to adapt as points are reassigned during iterations.
- **Incremental Update:**
    - Instead of recalculating centroid from scratch after all assignments:
        - Update centroid **incrementally** as each point is assigned.
        -

<u>Bisecting K-Means Clustering:</u>

- Bi-secting K-Means is a hierarchical variant of K-Means.
- Combines the simplicity of K-Means with a top-down divisive clustering approach.
- Useful for large datasets and reduces sensitivity to random initialization.

Algorithm Steps:

1. **Start with all data points in a single cluster.**
2. **Repeat until the desired number of clusters (k) is reached:**
    - Choose a cluster to split (usually the one with the **largest SSE**).
    - Apply **standard K-Means with k=2** (bi-section) to split it into two sub-clusters.
    - Keep the split that **minimizes total SSE** across clusters.
3. **Update cluster assignments** after each split.

<u>Hierarchical Clustering:</u>

While partitioning methods meet the basic clustering requirement of organizing a set of objects into a number of exclusive groups, in some situations we may want to partition our data into groups at different levels such as in a hierarchy. A hierarchical clustering method works by grouping data objects into a hierarchy or "tree" of clusters. Representing data objects in the form of a hierarchy is useful for data summarization and visualization. For example, as the manager of human resources at AllElectronics, you may organize your employees into major groups such as executives, managers, and staff. You can further partition these groups into smaller subgroups. For instance, the general group of staff can be further divided into subgroups of senior officers, officers, and trainees. All these groups form a hierarchy. We can easily summarize or characterize the data that are organized into a hierarchy, which can be used to find, say, the average salary of managers and of officers.

Consider handwritten character recognition as another example. A set of handwriting samples may be first partitioned into general groups where each group corresponds to a unique character. Some groups can be further partitioned into subgroups since a character may be written in multiple substantially different ways. If necessary, the hierarchical partitioning can be continued recursively until a desired granularity is reached.

5.3.1 Agglomerative versus Divisive Hierarchical Clustering

A hierarchical clustering method can be either agglomerative or divisive, depending on whether the hierarchical decomposition is formed in a bottom-up (merging) or top- down (splitting) fashion.

Let's have a closer look at these strategies.

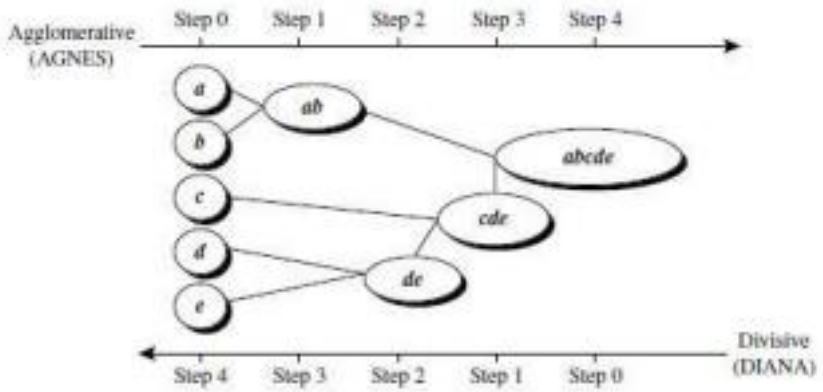
An agglomerative hierarchical clustering method uses a bottom-up strategy. It typically starts by letting each object form its own cluster and iteratively merges clusters into larger and larger clusters, until all the objects are in a single cluster or certain termination conditions are satisfied. The single cluster becomes the hierarchy's root. For the merging step, it finds the two clusters that are closest to each other (according to some similarity measure), and combines the two to form one cluster. Because two clusters are merged per iteration, where each cluster contains at least one object, an agglomerative method requires at most n iterations.

A divisive hierarchical clustering method employs a top-down strategy. It starts by placing all objects in one cluster, which is the hierarchy's root. It then divides the root cluster into several smaller subclusters, and recursively partitions those clusters into smaller ones. The partitioning process continues until each cluster at the lowest level is coherent enough—either containing only one object, or the objects within a cluster are sufficiently similar to each other.

In either agglomerative or divisive hierarchical clustering, a user can specify the desired number of clusters as a termination condition.

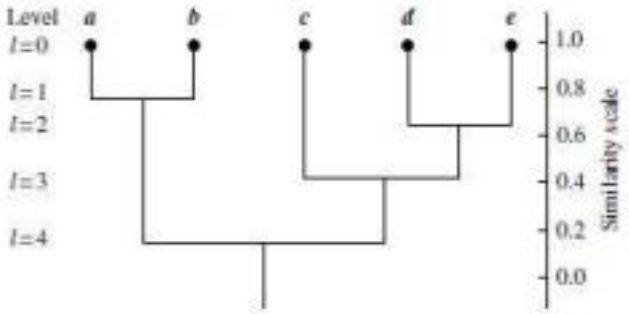
A tree structure called a dendrogram is commonly used to represent the process of hierarchical clustering. It shows how objects are grouped together (in an agglomerative method) or partitioned (in a divisive method) step-by-step. Figure 4 shows a dendrogram for the five objects presented in Figure 3, where L = 0 shows the five objects as singleton clusters at level 0. At L = 1, objects a and b are grouped together to form the first cluster, and they stay together at all subsequent levels. We can also use a vertical axis to show the similarity scale between clusters. For example, when the similarity of two groups of objects, {a, b} and {c, d, e} , is roughly 0.16, they are merged together to form a single cluster.

A challenge with divisive methods is how to partition a large cluster into several smaller ones. For example, there are 2^{n-1}-1 possible ways to partition a set of n objects into two exclusive subsets, where n is the number of objects. When n is large, it is computationally prohibitive to examine all possibilities. Consequently, a divisive method typically uses heuristics in partitioning, which can lead to inaccurate results. For the sake of efficiency, divisive methods typically do not backtrack on partitioning decisions that have been made. Once a cluster is partitioned, any alternative partitioning of this cluster will not be considered again. Due to the challenges in divisive methods, there are many more agglomerative methods than divisive methods.



Agglomerative and divisive hierarchical clustering on data objects {a, b, c, d, e}.

Figure 3



Dendrogram representation for hierarchical clustering of data objects $\{a, b, c, d, e\}$.

Figure 4

Distance Measures in Algorithmic Methods

Whether using an agglomerative method or a divisive method, a core need is to measure the distance between two clusters, where each cluster is generally a set of objects.

Four widely used measures for distance between clusters are as follows, where $|p - p'|$ is the distance between two objects or points, p and p'; m_i is the mean for cluster, C_i; and n_i is the number of objects in C_i. They are also known as *linkage measures*.

Minimum distance: $dist_{min}(C_i, C_j) = \min_{p \in C_i, p' \in C_j} \{|p - p'|\}$

Maximum distance: $dist_{max}(C_i, C_j) = \max_{p \in C_i, p' \in C_j} \{|p - p'|\}$

Mean distance: $dist_{mean}(C_i, C_j) = |m_i - m_j|$

Average distance: $dist_{avg}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p \in C_i, p' \in C_j} |p - p'|$

Mean distance is also called centroid distance. It is distance between two clusters centroids

Medoid Distance: distance between the medoids of two clusters, i.e.,

$dist(K_i, K_j) = dist(M_i, M_j)$

Medoid: a chosen, centrally located object in the cluster

When an algorithm uses the minimum distance, dmin(Ci, Cj), to measure the distance between clusters, it is sometimes called a nearest-neighbor clustering algorithm. Moreover, if the clustering process is terminated when the distance between nearest clusters exceeds a user-defined threshold, it is called a single linkage algorithm.

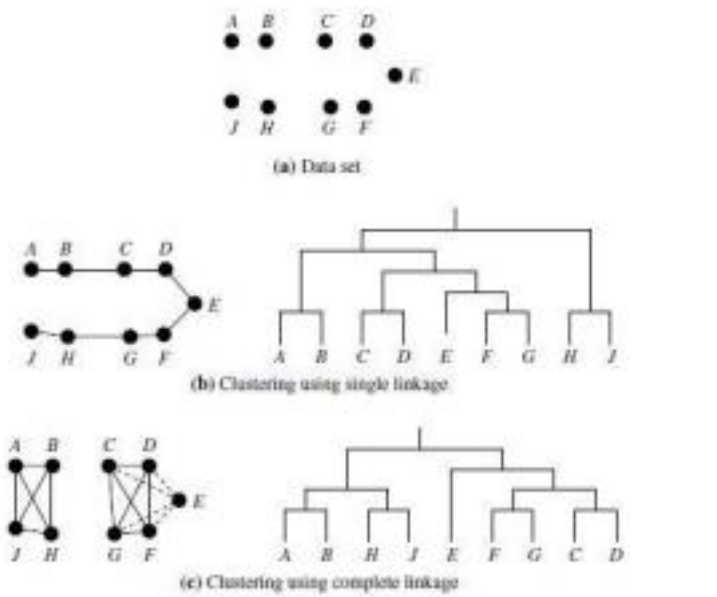
If we view the data points as nodes of a graph, with edges forming a path between the nodes in a cluster, then the merging of two clusters, Ci and Cj, corresponds to adding an edge between the nearest pair of nodes in Ci and Cj. Because edges linking clusters always go between distinct clusters, the resulting graph will generate a tree. Thus, an agglomerative hierarchical clustering algorithm that uses the minimum distance measure is also called a minimal spanning tree algorithm, where a spanning tree of a graph is a tree that connects all vertices, and a minimal spanning tree is the one with the least sum of edge weights.

When an algorithm uses the maximum distance, dmax(Ci, Cj), to measure the distance between clusters, it is sometimes called a farthest-neighbor clustering algorithm. If the clustering process is terminated when the

maximum distance between nearest clusters exceeds a user-defined threshold, it is called a complete-linkage algorithm. By viewing data points as nodes of a graph, with edges linking nodes, we can think of each cluster as a complete subgraph, that is, with edges connecting all the nodes in the clusters. The distance between two clusters is determined by the most distant nodes in the two clusters. Farthest-neighbor algorithms tend to minimize the increase in diameter of the clusters at each iteration. If the true clusters are rather compact and approximately equal size, the method will produce high-quality clusters. Otherwise, the clusters produced can be meaningless.

The previous minimum and maximum measures represent two extremes in mea suring the distance between clusters. They tend to be overly sensitive to outliers or noisy data. The use of mean or average distance is a compromise between the mini- mum and maximum distances and overcomes the outlier sensitivity problem. Whereas the mean distance is the simplest to compute, the average distance is advantageous in that it can handle categoric as well as numeric data. The computation of the mean vector for categoric data can be difficult or impossible to define.

Example: Single versus complete linkages. Let us apply hierarchical clustering to the data set of Figure 5 (a). Figure 5 (b) shows the dendrogram using single linkage. Figure 5 (c) shows the case using complete linkage, where the edges between clusters {A, B, J , H} and {C, D, G, F, E} are omitted for ease of presentation. This example shows that by using single linkages we can find hierarchical clusters defined by local proximity, whereas complete linkage tends to find clusters opting for global closeness.



Hierarchical clustering using single and complete linkages.

Density Based Methods

Partitioning and hierarchical methods are designed to find spherical-shaped clusters. They have difficulty finding clusters of arbitrary shape such as the “S” shape and oval clusters. Given such data, they would likely inaccurately identify convex regions, where noise or outliers are included in the clusters.

To find clusters of arbitrary shape, alternatively, we can model clusters as dense regions in the data space, separated by sparse regions. This is the main strategy behind density-based clustering methods, which can discover clusters of nonspherical shape.

Density based clustering comprises of a set of algorithms that, as the name suggests, uses the density of objects to assign cluster membership. The density

of an object o can be measured by the number of objects close to o. Areas of the feature space containing many (objects) packed closely together can be said to have high density. Areas of the feature space that contain few or no objects can be said to have low density. Distinct clusters in a dataset will be represented by regions of high density, separated by regions of low density. Density-based clustering algorithms attempt to learn these distinct regions of high density and partition them into clusters.

Major features:

• Discover clusters of arbitrary shape

• Handle noise

• One scan

• Need density parameters as termination condition

Examples:

DBSCAN, OPTICS, CLIQUE

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) finds core objects, that is, objects that have dense neighborhoods. It connects core objects and their neighborhoods to form dense regions as clusters.

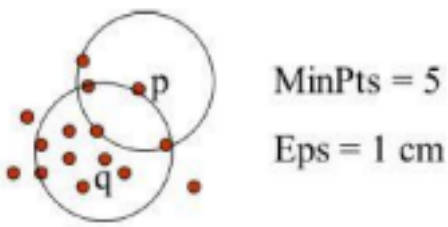
5.4.1 Density Based Clustering Basic Concepts

Two parameters

• *Eps*: Maximum radius of the neighbourhood

• Epsilon is simply how far away from each case (in an n-dimensional sphere) the algorithm will search for other cases around a point • *MinPts*: Minimum number of points in an Eps-neighbourhood of that point • *NEps(p)*: {q belongs to D | dist(p,q) ≤ Eps}

Directly density-reachable: A point *p* is directly density-reachable from a point *q* w.r.t. *Eps, MinPts* if *p* belongs to *NEps(q)*



core point condition:

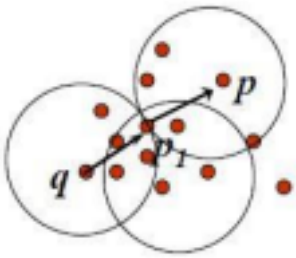
$|NEps\ (q)| \geq MinPts$

The *minPts* hyperparameter specifies the minimum number of points (cases) that a cluster must have in order for it to be a cluster

Density-reachable

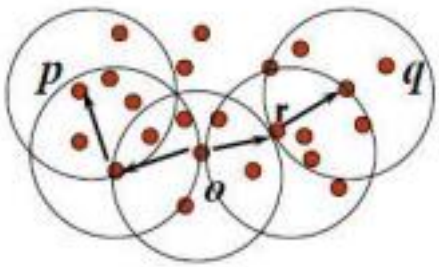
A point *p* is density-reachable from a point *q* w.r.t. *Eps, MinPts* if there is a chain of points *p1*, …, *pn, p1 =*

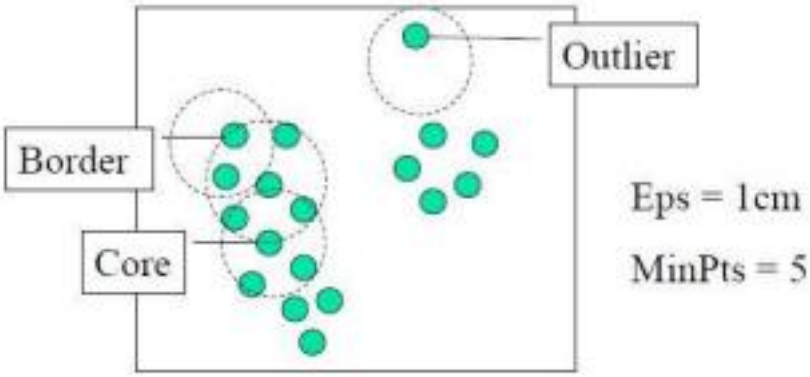
q, *pn* = *p* such that *pi+1* is directly density-reachable from *pi*



Density-connected

A point *p* is density-connected to a point *q* w.r.t. *Eps*, *MinPts* if there is a point *o* such that both, *p* and *q* are density-reachable from *o* w.r.t. *Eps* and *MinPts*





DBSCAN Algorithm

A cluster is defined as a maximal set of density-connected points. Discovers clusters of arbitrary shape in spatial databases with noise.

The density of an object o can be measured by the number of objects close to o.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) finds core objects, that is, objects that have dense neighborhoods. It connects core objects and their neighborhoods to form dense regions as clusters. A user-specified parameter $\varepsilon > 0$ is used to specify the radius of a neighborhood we consider for every object. The ε -neighborhood of an object o is the space within a radius ε centered at o. The density of a neighbourhood is no. of objects in the neighbourhood. To determine whether a neighborhood is dense or not, DBSCAN uses another user-specified parameter, MinPts, which specifies the density threshold of dense regions. An object is a core object if the ε -neighborhood of the object contains at least MinPts objects. Core objects are the pillars of dense

regions. Given a set, D, of objects, we can identify all core objects with respect to the given parameters, ε and MinPts. The clustering task is therein reduced to using core objects and their neighborhoods to form dense regions, where the dense regions are clusters.

Algorithm: DBSCAN: a density-based clustering algorithm.

Input:

- D: a data set containing n objects,
- ϵ: the radius parameter, and
- $MinPts$: the neighborhood density threshold.

Output: A set of density-based clusters.

Method:

```
mark all objects as unvisited;
do
    randomly select an unvisited object p;
    mark p as visited;
    if the ε-neighborhood of p has at least MinPts objects
        create a new cluster C, and add p to C;
        let N be the set of objects in the ε-neighborhood of p;
        for each point p' in N
            if p' is unvisited
                mark p' as visited;
                if the ε-neighborhood of p' has at least MinPts points,
                add those points to N;
            if p' is not yet a member of any cluster, add p' to C;
        end for
        output C;
    else mark p as noise;
until no object is unvisited;
```

<u>DBScan Algorithm Explanation :</u>

• Initially, all objects in a given data set D are marked as "unvisited."

• DBSCAN randomly selects an unvisited object p, marks p as "visited," and checks whether the ε - neighborhood of p contains at least MinPts objects.

• If not, p is marked as a noise point.

• Otherwise, a new cluster C is created for p, and all the objects in the ε - neighborhood of p are added to a candidate set, N.

• DBSCAN iteratively adds to C those objects in N that do not belong to any cluster.

• In this process, for an object p' in N that carries the label "unvisited," DBSCAN marks it as "visited" and checks its ε -neighborhood.

• If the ε -neighborhood of p' has at leastMinPts objects, those objects in the ε -neighborhood of p' are added to N.

• DBSCAN continues adding objects to C until C can no longer be expanded, that is, N is empty.

• At this time, cluster C is completed, and thus is output.

• To find the next cluster, DBSCAN randomly selects an unvisited object from the remaining ones.

• The clustering process continues until all objects are visitedDBScan Algorithm Explanation

- Initially, all objects in a given data set D are marked as "unvisited."
- DBSCAN randomly selects an unvisited object p, marks p as "visited," and checks whether the ε -neighborhood of p contains at least MinPts objects.
- If not, p is marked as a noise point.
- Otherwise, a new cluster C is created for p, and all the objects in the ε - neighborhood of p are added to a candidate set, N.
- DBSCAN iteratively adds to C those objects in N that do not belong to any cluster.
- In this process, for an object p' in N that carries the label "unvisited," DBSCAN marks it as

"visited" and checks its ε -neighborhood.

- If the ε -neighborhood of p' has at least MinPts objects, those objects in the ε -neighborhood of p' are added to N.
- DBSCAN continues adding objects to C until C can no longer be expanded, that is, N is empty.
- At this time, cluster C is completed, and thus is output.
- To find the next cluster, DBSCAN randomly selects an unvisited object from the remaining ones.
- The clustering process continues until all objects are visited DBSCAN is sensitive to parameters